TNV Hub Portal

Backend & Database Specification — Phase 1 Demo

Supabase (Postgres + Auth) | Internal dev-team document | Demo: Thu 22 Oct 2026

1. Architecture

Supabase provides Postgres, Auth, Storage and Edge Functions. The React app talks to Supabase directly. Row-level security (RLS) in Postgres is the access-control layer; the frontend only hides what the database already refuses.

Concern	Approach
Auth	Supabase Auth with email + password. Users choose a **username**; login resolves it to the email through a database function, then signs in with email + password.
Data	Postgres tables in section 3, all with RLS enabled.
Privileged actions	Edge Functions using the service key: send emails, record payments, approve applicants. The browser never holds the service key.
Email	Transactional email from an Edge Function (Resend or SMTP): welcome, approval, password reset (Supabase built-in).
Files	Storage bucket *receipts* and *photos*, private; signed URLs for download.
Roles	*profiles.role* plus an *is_admin()* SQL helper used by every policy.

Key decisions

- **One registration path, one table.** Every applicant is a row in *profiles*; status (*pending* / *approved*) and role are columns, not separate user types.
- **Do not email passwords.** The spec says credentials go by email, but applicants choose their own password. Send a welcome email with the username and a login link only.
- **Unpaid applicants stay.** Payment is never a condition for creating the profile or for entering the admin queue.
- **Admin-only data** (call logs, scores, notes) lives in separate tables with no applicant policy at all.
- **Aadhaar** is stored encrypted and shown masked (last 4) except for a Master Admin reveal action that is logged.

2. Entity overview

Table	Holds	Visible to
profiles	One row per person: identity, contact, district, status, role, TNV ID	Owner (own row), admins (all)
districts, wings, sectors	38 districts, wings (Administration, Online, ...), 13 sectors	Everyone (read)
whatsapp_links	Group links by district, wing, sector, plus announcements and social	Approved users, admins
payments	Membership payments and receipts	Owner, admins
donations	Open donations from Rs 1; may be anonymous	Admins (insert: anyone)
call_logs	Each call attempt: outcome, callback time, note, admin	Admins only
interview_questions	About 10 questions, editable, ordered	Admins only
interview_scores	Score per question per applicant (0 / 0.5 / 1)	Admins only
verifications	Final decision: total score, recommended role, approved by, time	Admins; owner reads outcome via profile

Table	Holds	Visible to
notifications	In-app alerts to admins (new registration)	Admins
audit_log	Sensitive actions: Aadhaar reveal, approvals, role changes	Master Admin

Relationships: profiles 1-N call_logs, 1-N interview_scores, 1-N payments, 1-1 verifications; profiles N-1 districts; interview_scores N-1 interview_questions; whatsapp_links N-1 districts / wings / sectors.

3. Database schema (SQL)

Run in order in the Supabase SQL editor. Types and names are final for Phase 1; add columns later, not renames.

3.1 Types and reference tables

```
create type app_status   as enum ('pending','approved');
create type app_role as enum ('volunteer','district_coordinator','wing_lead',
                              'dept_admin','master_admin');
create type call_outcome as enum ('no_answer','scheduled','attended');
create type pay_status   as enum ('paid','failed');

create table districts (id serial primary key, name text unique not null);   -- 38 rows
create table wings     (id serial primary key, name text unique not null);   -- Administration, Online, ...
create table sectors   (id serial primary key, name text unique not null);   -- 13 rows
```

3.2 Profiles

```
create table profiles (
  id              uuid primary key references auth.users(id) on delete cascade,
  username        text unique not null check (username ~ '^[A-Za-z0-9_]{4,20}$'),
  full_name       text not null,
  email           text not null,
  phone           text not null check (phone ~ '^[6-9][0-9]{9}$'),
  address         text,
  district_id     int references districts(id) not null,
  aadhaar_enc     text not null,            -- encrypted; never selected by client code
  aadhaar_last4   char(4) not null,         -- for masked display
  status          app_status not null default 'pending',
  role            app_role,                 -- null until verified
  wing_id         int references wings(id),
  tnv_id          text unique,              -- generated on approval
  photo_path      text,
  membership_paid boolean not null default false,
  created_at      timestamptz not null default now()
);

create function is_admin() returns boolean language sql stable security definer as $$
  select exists (select 1 from profiles
                 where id = auth.uid() and role in ('master_admin','dept_admin')) $$;
```

3.3 Payments, donations, call log, scoring, verification

```
create table payments (
  id uuid primary key default gen_random_uuid(),
  profile_id uuid not null references profiles(id),
  amount_inr int not null check (amount_inr = 365),
  status pay_status not null, gateway_ref text, receipt_no text unique,
  paid_at timestamptz default now()
);
create table donations (
  id uuid primary key default gen_random_uuid(),
  donor_name text, message text, amount_inr int not null check (amount_inr >= 1),
  gateway_ref text, created_at timestamptz default now()
);
```

```
create table call_logs (
  id uuid primary key default gen_random_uuid(),
  profile_id uuid not null references profiles(id) on delete cascade,
  admin_id uuid not null references profiles(id),
  outcome call_outcome not null,
  callback_at timestamptz,                  -- required when outcome = 'scheduled'
  note text, created_at timestamptz default now(),
  check (outcome <> 'scheduled' or callback_at is not null)
);
create table interview_questions (
  id serial primary key, position int not null, text text not null, active boolean default true
);
create table interview_scores (
  profile_id uuid references profiles(id) on delete cascade,
  question_id int references interview_questions(id),
  score numeric(2,1) not null check (score in (0, 0.5, 1)),
  note text, scored_by uuid references profiles(id),
  primary key (profile_id, question_id)
);
create table verifications (
  profile_id uuid primary key references profiles(id) on delete cascade,
  total_score numeric(3,1) not null, recommended_role app_role not null,
  approved_by uuid not null references profiles(id), approved_at timestamptz default now()
);
```

3.4 Links, notifications, audit

```
create table whatsapp_links (
  id serial primary key,
  kind text not null check (kind in ('district','wing','sector','announcements','social')),
  district_id int references districts(id), wing_id int references wings(id),
  sector_id int references sectors(id), label text not null, url text not null
);
create table notifications (
  id uuid primary key default gen_random_uuid(), kind text not null,
  profile_id uuid references profiles(id), read boolean default false, created_at timestamptz default now()
);
create table audit_log (
  id bigserial primary key, actor uuid references profiles(id), action text not null,
  target uuid, detail jsonb, at timestamptz default now()
);
```

4. Row-level security

Enable RLS on every table. The principle: deny by default, then allow the minimum.

Table	Applicant (own row)	Admin
profiles	select own; update only name, phone, address, photo. Cannot change status, role, tnv_id, aadhaar columns.	select / update all
payments	select own	select all
call_logs, interview_*, verifications, audit_log, notifications	**No access.** No policy exists for applicants.	select / insert / update
whatsapp_links	select only if profile status = approved	all
districts, wings, sectors	select	select
donations	insert via Edge Function only	select

Rules that must hold

- The **Aadhaar columns are never returned to the browser** for applicants or normal queries. Expose a *profiles_public* view without them, and reveal through the *reveal_aadhaar* function only.
- Applicants cannot self-approve: status and role are writable only by the *approve_applicant* function.
- dept_admin rows are created later by the Master Admin. For Thursday only the master_admin exists, and the policies already allow adding dept admins without schema change.

5. API surface

Plain table reads and writes go through supabase-js under RLS. The actions below are server-side functions because they cross trust boundaries.

Name	Kind	Purpose	Task
register	Edge Function	Validate, create auth user + profile (encrypt Aadhaar), insert admin notification, send welcome email. Returns session.	T2
username_available(u)	RPC	True if username is free	T2
login_email_for(username)	RPC (anon)	Returns the email for a username so the client can sign in; returns nothing for unknown names	T2
record_payment	Edge Function	Verify gateway result (or mock), insert payment, set membership_paid, generate receipt_no and PDF	T3
create_donation	Edge Function	Validate amount >= 1, take payment, insert donation	T12
log_call	RPC (admin)	Insert call_logs row; rule: scheduled needs callback_at	T6
save_scores	RPC (admin)	Upsert scores for an applicant; each must be 0, 0.5 or 1	T7
approve_applicant	Edge Function (admin)	Check all active questions scored; sum score; set role, status = approved; generate tnv_id; write verification; email approval; audit	T7, T8
reveal_aadhaar(id)	RPC (master)	Return decrypted Aadhaar; write audit row	T5
my_links()	RPC	For an approved user: own district link, wing link, all sector links, announcements, social	T10

6. Business rules

- **Status:** new profile = pending; only *approve_applicant* moves it to approved. No path back in Phase 1.
- **Call log:** unlimited attempts per applicant; the latest outcome is the applicant's current call status in the queue.
- **Scoring:** total = sum of question scores; max equals the number of active questions. Role recommendation is a suggestion; the admin's choice is saved.
- **TNV ID:** generated at approval. Format is pending confirmation; use *TNV-<district code>-<running number>* as placeholder.
- **Payment:** amount fixed at Rs 365. Failure never blocks registration. Receipt number format *TNV-R-<year>-<running number>*.
- **Follow-up:** a daily query lists scheduled callbacks that are due and no-answer applicants older than 24 hours (admin queue filter is enough for Thursday).

7. Seed data for the demo

- 38 Tamil Nadu districts, the wings (Administration, Online), 13 sectors. Ask Sir for the sector names.
- WhatsApp links: real links if available, otherwise clearly fake placeholders. At least three districts must differ.
- About 10 interview questions copied from the TNV Leadership Selection Portal.
- One Master Admin account for Sir, plus three test applicants: one paid, one unpaid, one already approved.

8. Backend checklist

Build items first, then the security items. Do not skip the security items for the demo; they are quick and protect real Aadhaar data.

Build

Done	Check	Ref
☐	Supabase project created; environment keys stored outside the repo	Setup
☐	All enums and tables from section 3 created without errors	Schema
☐	38 districts, wings, 13 sectors, interview questions and WhatsApp links seeded	Seed
☐	register creates auth user, profile and admin notification in one call	T2
☐	Duplicate username or duplicate email is rejected with a clear message	T2
☐	Login by username works; wrong username and wrong password give the same generic error	T2
☐	Password reset email arrives and the new password works	T2
☐	record_payment stores payment, sets membership_paid and returns a receipt; failed payment changes nothing else	T3
☐	Unpaid applicant still appears in the admin queue	T3, T5
☐	log_call saves all three outcomes; scheduled without a time is rejected	T6
☐	save_scores accepts 0, 0.5, 1 and rejects every other value	T7
☐	approve_applicant refuses when any active question is unscored	T7
☐	approve_applicant sets status, role and TNV ID, writes verification and sends the approval email	T8
☐	my_links returns the right district link for users in three different districts	T10

Security

Done	Check	Ref
☐	RLS enabled on every table; a table without policies returns nothing	RLS
☐	As an applicant: cannot read other profiles, call logs, scores or verifications	RLS
☐	As an applicant: cannot update own status, role or tnv_id (try it directly)	RLS
☐	As a pending applicant: whatsapp_links returns zero rows	RLS
☐	Aadhaar never appears in any applicant or list query; reveal writes an audit row	T5
☐	Service key exists only in Edge Function secrets, never in the frontend	Setup
☐	Database backup or point-in-time recovery turned on before real data is entered	Ops

9. End-to-end flow test

Run this full script twice before the demo, once as Sir will see it and once in a private window on a phone. Every step must pass without touching the database by hand. Test applicant: use a new username each run.

Done	Check	Ref
☐	1. Open the landing page; Register as Volunteer opens the form	T1
☐	2. Submit the form with a bad Aadhaar (11 digits): blocked with a clear message	T2
☐	3. Submit valid details: account created, welcome email arrives with username and login link, no password in it	T2
☐	4. Payment step: choose Skip. Dashboard shows only the "under review" message	T3, T4
☐	5. Log out and log in again with username and password: still the pending screen	T4
☐	6. Try /admin and a verified-only URL as this applicant: refused	Guards
☐	7. Log in as Master Admin: notification for the new registration; applicant listed as unpaid	T5
☐	8. Open the applicant, log *No Answer*: appears in history; queue filter shows it	T6
☐	9. Log *Scheduled* with a callback time; then log *Attended*	T6
☐	10. Score all questions using 0, 0.5 and 1; total is correct; pick a role	T7
☐	11. Mark Verified: confirmation, status becomes Approved, approval email sent	T8
☐	12. Applicant's open dashboard unlocks within a minute (or after refresh)	T8
☐	13. ID card and appointment letter show the right name, role, district and TNV ID; both download	T9
☐	14. WhatsApp section shows that applicant's own district link, wing, 13 sectors, announcements	T10
☐	15. Repeat steps 3 to 14 with a paid applicant: receipt downloads from the profile	T3
☐	16. Donate Rs 1 from the donation page while signed out: recorded; not visible to applicants	T12